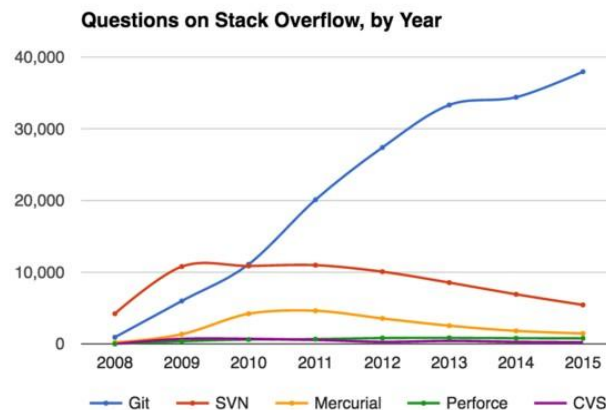


Krótkie wprowadzenie do systemu kontroli wersji Git

Obecnie najpopularniejszym systemem kontroli wersji jest system Git, który praktycznie zdominował rynek i na opisie podstawowych funkcji którego zajmiemy się na tych zajęciach. Na wykresie powyżej pokazano ilość zapytań odnośnie poszczególnych systemów kontroli wersji w serwisie Stack Overflow na przestrzeni kilku lat.



Oprócz tego systemu istnieją też inne systemy kontroli wersji np. SVN lub Mercurial, jednakże ich znaczenie powoli staje się marginalne. Poniżej przedstawię dane dotyczące popularności systemów kontroli wersji w roku 2025 (na podstawie ankiety szczegóły tutaj:

<https://rhodecode.com/blog/156/version-control-systems-popularity-in-2025>

oraz tutaj

<https://stackoverflow.blog/2023/01/09/beyond-git-the-other-version-control-systems-developers-use/>



I) Dlaczego warto stosować system kontroli wersji ?

Najpierw odpowiem na to pytanie w kontekście zagadnień, które omawiamy na naszych zajęciach. Otóż niedługo będziemy na ćwiczeniach omawiać zagadnienia związane z refaktoryzacją kodu. Niestety, wszelkie modyfikacje już istniejącego kodu (szczególnie w bardziej złożonych projektach) czasami mogą prowadzić do różnego rodzaju błędów. Dlatego dobrze mieć możliwość łatwego przywrócenia wersji kodu sprzed modyfikacji. Oczywiście można starać się panować nad tym ręcznie (np. w osobnym folderze trzymać wersje projektu sprzed refaktoryzacji) i w razie potrzeby wracać do niej, jednakże przy dużych projektach, dużej liczbie zmian, czy też dużej liczbie osób pracujących nad projektem jest to skrajnie niewygodne. Lepiej w tym celu stosować system kontroli wersji. **Jedną z głównych zalet systemu kontroli wersji jest możliwość łatwego przywrócenia projektu (np. plików z kodami źródłowymi) do jednej z wcześniejszych wersji.** Możliwość przywrócenia wcześniejszej wersji projektu dobrze współgra z prowadzeniem projektu zgodnie z podejściem iteracyjnym lub przyrostowym (stosowanym w metodach AGILE np. w SCRUMie).

Stosowanie systemu kontroli wersji (oprócz możliwości przywrócenia projektu do jednego z wcześniejszych stanów) niesie wiele innych korzyści. Poniżej wymienię najważniejsze cechy systemu Git.

- 1) **Umożliwia tworzenie tzw. gałęzi (ang. branches).** Chodzi o to, że gdy rozwijamy już istniejący projekt (zwłaszcza duży) wygodnie mieć przynajmniej dwie wersje projektu: wersję „produkcyjną”, która aktualnie udostępniona jest klientowi oraz wersję developerską, w której dodajemy nowe funkcjonalności, które dopiero za jakiś czas (po testach) udostępniemy klientowi. Co istotne na bieżąco trzeba dokonywać zmian w obu wersjach (np. naprawiać krytyczne błędy zauważone przez użytkowników w wersji produkcyjnej oraz ustawicznie dodawać nowe funkcje w wersji developerskiej). Ważne jest też to, że kiedyś te dwie gałęzie trzeba jakoś scalić. Proszę pamiętać, że obie wersje projektów mają swoje własne zmiany, zatem proste zastąpienie plików w gałęzi produkcyjnej plikami z gałęzi developerskiej może nie wystarczyć (można pominąć wtedy zmiany, dokonywane równolegle w gałęzi produkcyjnej, tzn. np. wspomniane poprawki błędów krytycznych). **System kontroli wersji Git umożliwia tworzenie takich gałęzi oraz znacząco ułatwia ich scalanie.**
- 2) **W projektach nad którymi pracuje wielu programistów system Git umożliwia scalanie ich pracy.** System Git może pracować całkowicie „lokalnie” na jednym komputerze (jest to sensowne przy projektach prowadzonych przez jedną osobę). Może także korzystać ze zdalnego repozytorium (np. github, bitbucket lub gitlab). W takiej sytuacji programiści pracują na lokalnych kopiach projektu (dokonują w nich poprawek, tworzą nowe wersje, które są widoczne tylko dla nich) i co jakiś czas mogą wgrywać swoje ulepszone wersje do zdalnego repozytorium by widzieli je inni programiści (oczywiście wtedy mogą się pojawić kwestie konfliktów, gdy dany fragment kodu został w różny sposób zmodyfikowany przez dwie osoby, ale system Git ma narzędzia, by sobie z tym radzić.

II) Instalacja, interfejs GUI.

Strona domowa Git-a: <https://git-scm.com/>

System ten jest wolnym oprogramowaniem opartym na licencji GNU General Public License (w wersji bodajże 2.0)

Podstrona z której można go pobrać:

<https://git-scm.com/downloads>

Interfejs graficzny

Generalnie Git ma interfejs konsolowy (wydaje mi się, że korzysta z niego zdecydowana większość programistów). Istnieje jednak nakładka graficzna na ten system:

<https://www.sourcetreeapp.com/>

Jest ona rozwijana przez przedsiębiorstwo Atlassian (kojarzone z systemem zarządzania projektami JIRA czy systemem pracy grupowej Confluence).

III) Podstawowe pojęcia

Rewizja (revision) – pojedynczy zapisany stan projektu (zbioru śledzonych plików). Mówiąc wprost można odtworzyć stan projektu zapisany w dowolnej rewizji.

Working tree – folder, który jest powiązany z pojedynczym repozytorium Git-a. Zawiera on pliki i/lub podfoldery, projektu. To, że pliki leżą w working tree nie oznacza, że automatycznie są śledzone (zapisywane w rewizjach).

Staging area -pliki/katalogi, których stan zostanie zapisany w kolejnej rewizji

Podstawowa obsługa

(pod Windows, pod Linuxem jest podobnie)

1) Inicjalizacja repozytorium lokalnego

Założmy, że mamy pusty folder, w którym będziemy trzymać pliki projektu. Aby utworzyć w nim repozytorium Git-a, w linii komend przechodzimy do tego folderu (komenda **cd** i ścieżka do folderu) i następnie dajemy **git init**

Po wykonaniu tego polecenia wyświetli się w konsoli komunikat: „Initialized empty Git repository in C:/Users/tomek/Desktop/GitTesty/.git/”

W folderze zostanie też utworzony ukryty folder o nazwie `.git` . Jeśli chcemy używać Git-a w danym projekcie to tego foldera nie wolno usuwać, gdyż w nim Git trzyma wszystkie dane.

2) Dodanie plików do staging area.

W początkowo pustym folderze utworzymy sobie dowolne pliki. Git obsługuje dowolne pliki, mogą być to rysunki, biblioteki `.dll` itd. Ale najczęściej Git-a używa się do wersjonowania plików tekstowych (np. kodów źródłowych). Utwórzmy zatem dwa pliki tekstowe `.txt` (jeden pusty, drugi z treścią).

Teraz dodajmy wszystkie pliki z folderu do śledzenia robi się to poleceniem:

`git add -A` (ważne jest by `A` było duże).

Git nic nam nie wypisze, ale od teraz jesteśmy gotowi do tworzenia rewizji „projektu”.

UWAGA. Przed stworzeniem każdej rewizji trzeba wskazać, które pliki w niej uwzględnić (czyli np. skorzystać z **`git add -A`**).

3) Tworzenie rewizji (czyli zapisu stanu projektu, który później można odtworzyć) Robi się to poleceniem

`git commit -m „tutaj dodajemy komentarz – np. opis wprowadzonych zmian”` po parametrze `-m` dodajemy (np. w cudzysłowie) komentarz opisujący daną rewizję projektu. *PS. Uwaga, proszę nie próbować kopiować cudzysłowia z tego tekstu powyżej (to inny znak niż cudzysłów wpisywany z klawiatury i polecenie prawie na pewno nie zadziała wywalając jakiś błąd).* Gdy polecenie wykona się poprawnie dostaniemy komunikat w rodzaju

```
C:\Users\tomek\Desktop\GitTesty>git commit -m "pierwsza wersja projektu"
[master (root-commit) fc60e13] pierwsza wersja projektu
 2 files changed, 5 insertions(+)
 create mode 100644 plik1.txt
 create mode 100644 plik2.txt

C:\Users\tomek\Desktop\GitTesty>
```

Teraz proszę zmodyfikować jeden z plików tekstowych, wykonać **`git add -A`** aby dodać pliki do staging area i zapisać poleceniem **`commit`** nową rewizję.

4) Wyświetlanie spisu istniejących rewizji

Aby wyświetlić spis aktualnych rewizji trzeba skorzystać z polecenia **`git log`**

Wyświetli się wtedy poniższa lista rewizji (im wyżej na liście tym późniejsza wersja).

```

C:\Users\tomek\Desktop\GitTesty>
C:\Users\tomek\Desktop\GitTesty>git log
commit 6f65a8e015dfc6081eaad018f321d735677ff47e (HEAD -> master)
Author: Tomek U <tomek_urbanczyk@op.pl>
Date:   Sun Jan 2 20:01:11 2022 +0100

    druga wersja

commit 73e11d33dde928b12bdc6416a0d46f03e43b2f7d
Author: Tomek U <tomek_urbanczyk@op.pl>
Date:   Sun Jan 2 20:00:31 2022 +0100

    Pierwsza wersja

C:\Users\tomek\Desktop\GitTesty>

```

Na żółto (po słowie commit) program wypisał hashe poszczególnych rewizji (mogą być przydatne przy przywracaniu projektu do wcześniejszego stanu). Po żółtym tekście jest wypisany autor rewizji, data jej stworzenia oraz message (jeśli był podany), który wpisano (po -m) przy tworzeniu rewizji. Uwaga1. Istnieje wersja polecenia, która wypisze listę bardziej kompaktowo

git log --pretty=oneline

```

C:\Users\tomek\Desktop\GitTesty>git log --pretty=oneline
6f65a8e015dfc6081eaad018f321d735677ff47e (HEAD -> master) druga wersja
73e11d33dde928b12bdc6416a0d46f03e43b2f7d Pierwsza wersja

C:\Users\tomek\Desktop\GitTesty>

```

Uwaga2. Gdy projekt ma dużo rewizji, to **git log** wyświetli kilka ostatnich (chyba 5) gdy damy enter to wypiszą się kolejne. Przy długich projektach przeklikanie enterem wszystkich może być kłopotliwe, aby przerwać wystarczy wcisnąć klawisz **q**.

5) Przywrócenie poprzedniej wersji projektu

Przyjmujemy teraz, że nie mamy w projekcie żadnych dodatkowych gałęzi. Aby przywrócić projekt do jednej z poprzednich rewizji można użyć polecenia

git checkout xxxxx gdzie **xxxxx** to pierwszych kilka znaków hashu rewizji (można odczytać z rezultatów **git log**)

Git checkout przywróci poprzednie wersje plików, używając tego polecenia trzeba mieć na uwadze że git przechodzi w stan detached HEAD.

Oznacza to, że nowe commity robione w tym stanie nie będą przypisane do żadnej gałęzi.

Innym sposobem na przywrócenie projektu jest zastosowanie polecenia **git revert xxxxx** Polecenie to zapisuje jako kolejny commit jeden z poprzednich commitów (kopiuje go).

Nowemu commitowi trzeba nadać nowy opis. W tym celu git wyświetli edytor, który obsługuje się jak VIM w Linuksie

Aby dodać nazwę najpierw trzeba wcisnąć klawisz „i” (od insert). Wtedy edytor przejdzie w tryb wprowadzania znaków. Następnie nadaje się nazwę commitowi. Całość zatwierdza się klikając kolejno klawisz escape, dwukropek, literę w (write) literę q (quit) oraz klawisz enter. Gdy wystarczy nam „domyślna” nazwa rewizji nadana przez git to wystarczy wpisać w, q i enter.

W powyższym przykładzie miałem dwie gałęzie: gałąź master oraz gałąź „nowa_galez”.

Aktualna gałąź oznaczona jest gwiazdką oraz zieloną czcionką. Aby zmienić gałąź stosuje się polecenie ***git checkout nazwa_galezi***


```
C:\Users\tomek\Desktop\GitTesty>git checkout nowa_galaz
Previous HEAD position was 9476381 przywrocona wersja "wersja 2"
Switched to branch 'nowa_galaz'

C:\Users\tomek\Desktop\GitTesty>git branch
  master
* nowa_galaz

C:\Users\tomek\Desktop\GitTesty>
```

Gdy wprowadzamy zmiany do jednej gałęzi inne pozostają nienaruszone (nowo dodane commity są widoczne tylko w bieżącej gałęzi). Można to sprawdzić dodając w nowej gałęzi commita, wypisując commita (nowy będzie widoczny) przechodząc (stosując git checkout) do gałęzi master i znów wypisując commita (nowego nie będzie).

```
C:\Users\tomek\Desktop\GitTesty>git branch
  master
* nowa_galaz

C:\Users\tomek\Desktop\GitTesty>git add -A

C:\Users\tomek\Desktop\GitTesty>git commit -m "dodano plik4.txt"
[nowa_galaz f661444] dodano plik4.txt
1 file changed, 1 insertion(+)
create mode 100644 plik4.txt

C:\Users\tomek\Desktop\GitTesty>git log --pretty=oneline
f661444a7201bfcba012ac941962d65e90871953 (HEAD -> nowa_galaz) dodano plik4.txt
cb3e02f90976bdc36b48ad7b86489e65287d60d5 (master) Przywróć commit Revert "wersja 4"
4570ccf86020b0fd33397ad49889e6d23162d8bb wersja 5
ca946eba433fc2b8cdbaa3a2277e8a7696630aef wersja 4
9476381d7a4f54272ccec5186045f953ebf9f6d0 przywrocona wersja "wersja 2"
c99e2091875eaf83cd845eea5ad3fa44156a29e2 wersja 3
0d91b499170c053009f20c52fb57dc5e41f79d5a wersja 2
6c9819f911c6b29b57415efbf864f96ac7fddff2 wersja 1

C:\Users\tomek\Desktop\GitTesty>git checkout master
Switched to branch 'master'

C:\Users\tomek\Desktop\GitTesty>git log --pretty=oneline
cb3e02f90976bdc36b48ad7b86489e65287d60d5 (HEAD -> master) Przywróć commit Revert "wersja 4"
4570ccf86020b0fd33397ad49889e6d23162d8bb wersja 5
ca946eba433fc2b8cdbaa3a2277e8a7696630aef wersja 4
9476381d7a4f54272ccec5186045f953ebf9f6d0 przywrocona wersja "wersja 2"
c99e2091875eaf83cd845eea5ad3fa44156a29e2 wersja 3
0d91b499170c053009f20c52fb57dc5e41f79d5a wersja 2
6c9819f911c6b29b57415efbf864f96ac7fddff2 wersja 1
```

7) Git Status

Polecenie git status jest często bardzo przydatne (zwłaszcza gdy coś nie działa). Gdy wszystko jest OK powinno wyświetlić mniej więcej to:

```
C:\Users\tomek\Desktop\GitTesty>git status
On branch master
Your branch is up to date with 'origin/master'.

nothing to commit, working tree clean
```

(bez podpięcia repozytorium zdalnego wyświetli się bez linijki „Your branch is up to date with ‘origin/master’”)

8) Tagi

Ważne miejsca w projekcie (do których chcielibyśmy wrócić w łatwiejszy sposób niż poprzez hasha commita) można oznaczyć za pomocą etykiety. Oznaczyć można dowolny commit, albo aktualny albo historyczny (wtedy trzeba do niego przejść używając **git checkout xxx**) Aby dodać etykietę należy skorzystać z polecenia **git tag abc** gdzie abc to nazwa etykiety. Od tej pory można korzystać z nadanej etykiety zamiast z hasha (działa np. z checkoutem).

Zadania.

- 1) Proszę sobie zainstalować git-a utworzyć lokalne repozytorium i dodać do niego kilka commitów.
- 2) Proszę zrobić zadanie z „Pegaza”
- 3) Jak ktoś chce to proszę sobie zainstalować sourcetree i spróbować otworzyć w nim projekt git (trzeba wskazać wybrany folder). Najlepiej by projekt miał więcej niż jedną gałąź (interfejs wygląda jak niżej).

